# MEDIA DIVERSITY WATCH

Autonomous Media Inclusivity Research Agent

**Project 3 · Reference Guide & Reflection Report**

Al Jazeera English · The Guardian · NPR · New York Times ·

Dina Bosma-Buczynska

## 1. Use Case and Organization.

Media Diversity Watch is a fictional media accountability NGO. Their mission: track whether major news outlets are improving or declining on inclusive coverage, and publish evidence-based reports that create public pressure for change.

Nova Tracks is a media diversity researcher who monitors four major outlets every week and needs scored, sourced reports without spending days manually searching news sources.

| Field | Value |
|---|---|
| Outlets | Al Jazeera English, The Guardian, NPR, New York Times |
| Schedule | Every Monday at 6am (automated) + on-demand form |
| Delivery | Notion database: score, week number, harm flags, full report |
| Evaluation | 4 angles × 4 communities per outlet, every week |

## 2. Outlet Selection: Changes Made and Why.

Two outlets were changed from the original plan. These were deliberate analytical decisions, not workarounds.

### BBC → Al Jazeera English

BBC's RSS feed publishes articles under editorial desk names rather than individual journalist bylines. Al Jazeera English was selected instead because it brings a non-Western, Global South editorial perspective: a meaningful contrast to the three Western outlets. After testing, AJE also has no RSS bylines, but this was documented as a transparency signal rather than a gap.

> *→ The absence of bylines IS the finding. The agent surfaces and documents it under Angle 1, which is more sophisticated than simply excluding outlets that lack bylines.*

### Reuters → New York Times

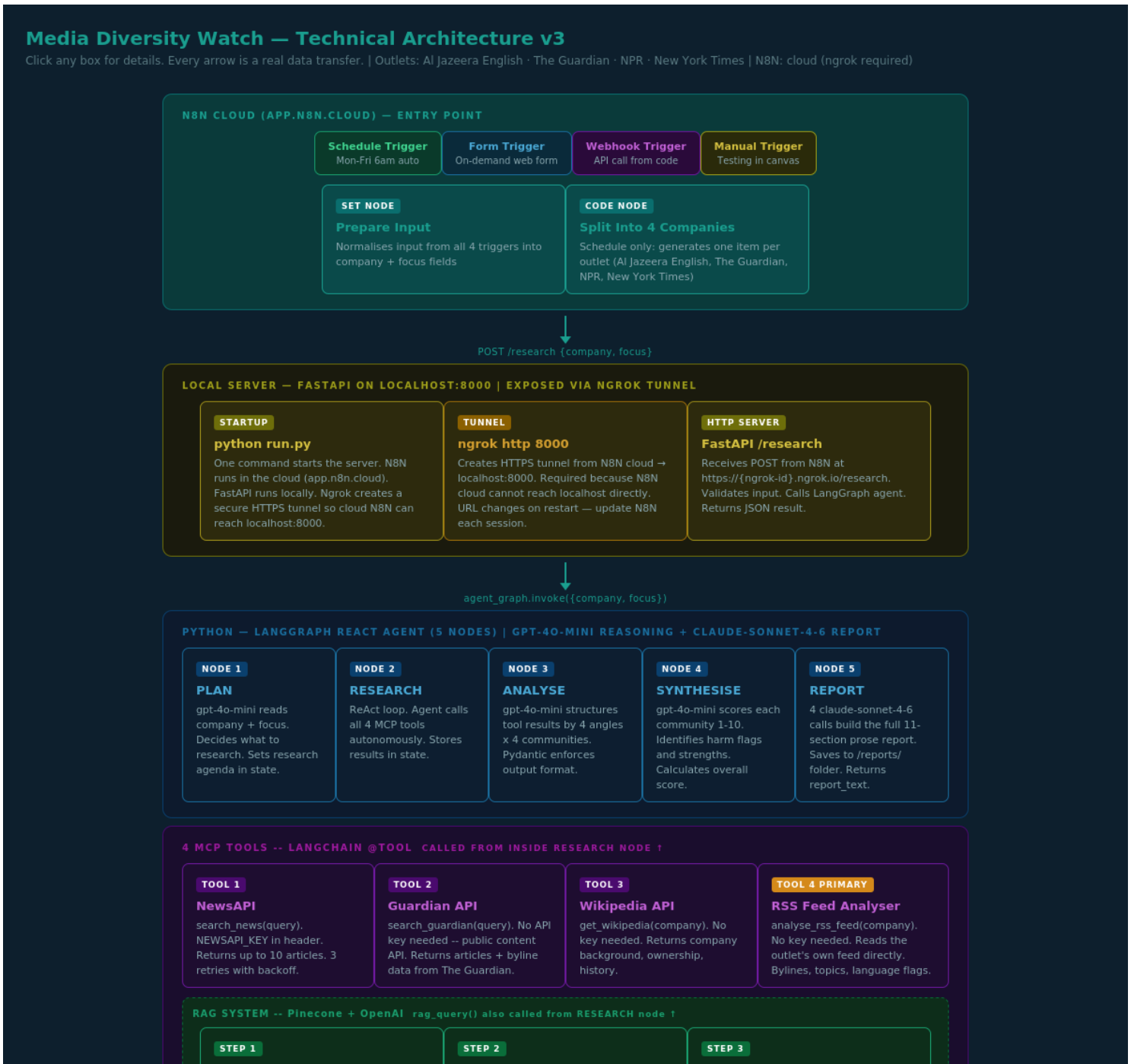
Reuters discontinued their public RSS feeds around 2020. A workaround routing through Google News was noisy and indirect. The New York Times has a clean official RSS feed with named bylines on most articles. NYT also represents the US liberal newspaper-of-record tradition: a useful benchmark.

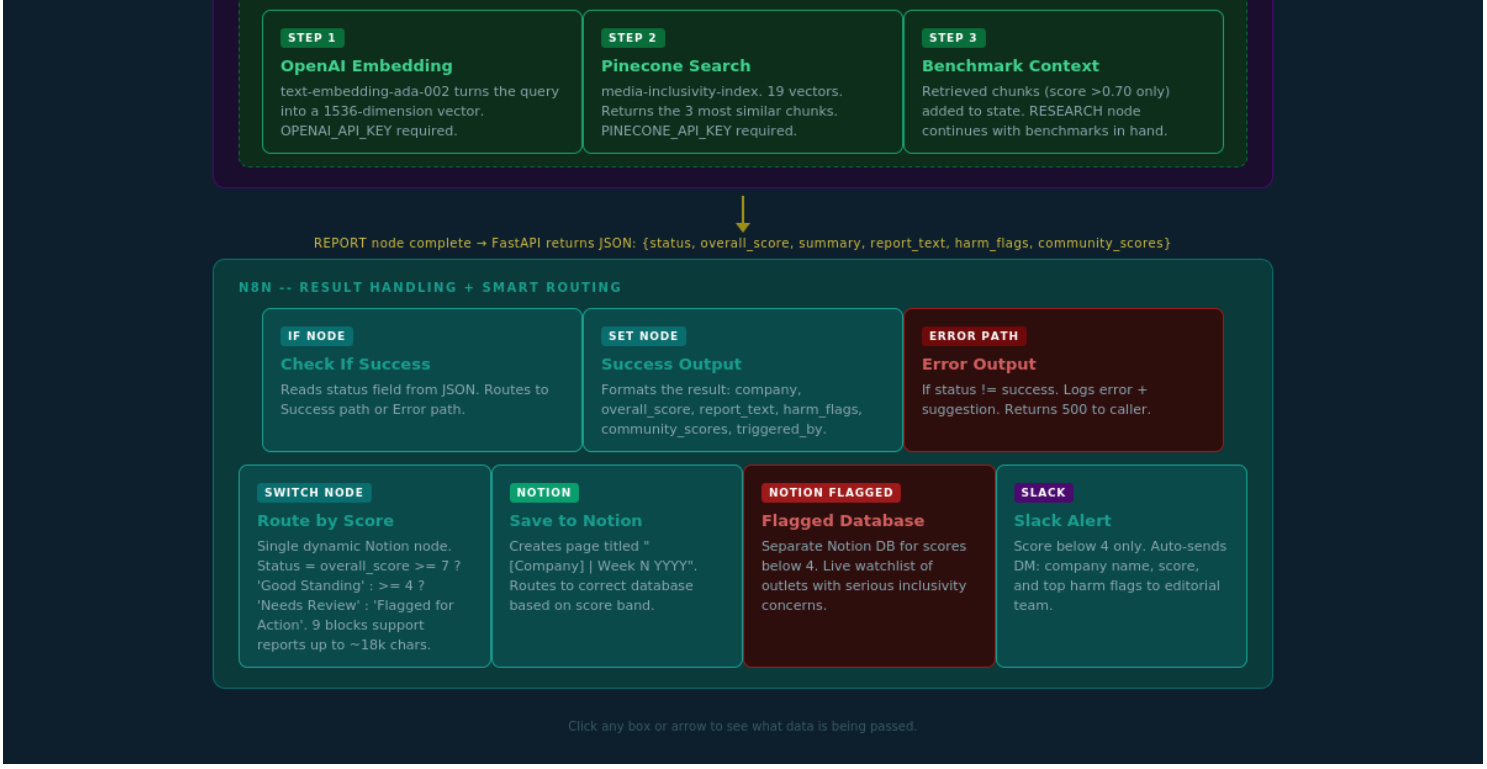
| Outlet | Tradition | RSS Bylines | Why Included |
|---|---|---|---|
| Al Jazeera English | Global South (Qatar) | No (editorial desk) | Non-Western perspective; government-funded, editorially independent |

| Outlet | Tradition | RSS Bylines | Why Included |
|---|---|---|---|
| The Guardian | UK progressive broadsheet | Yes (30/30 named) | Named bylines + Guardian Open Platform API for full article text |
| NPR | US public radio/digital news | Yes (27/30 named) | 5 topic feeds → 30 articles; Race feed directly on-topic |
| New York Times | US newspaper of record | Yes (25/25 named) | Strong archive; US liberal mainstream benchmark; some paywall limits |

### 3. Technical Architecture

The full system connects N8N Cloud (automation layer) → ngrok tunnel → FastAPI local server → LangGraph agent → 4 MCP tools + RAG → back to N8N → Notion + Slack.





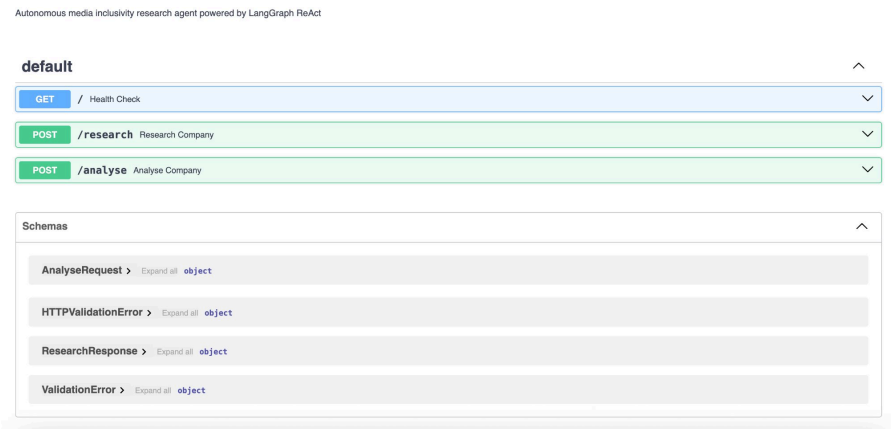

### Stack at a Glance

| Layer | Technology | Why |
|---|---|---|
| Orchestration | N8N Cloud (app.n8n.cloud) | Schedule, form, webhook, manual triggers in one canvas |
| Tunnel | ngrok | Bridges cloud N8N to local FastAPI: Required (see section 5.) |
| API Server | FastAPI + uvicorn | Validates input, serves JSON, auto-generates Swagger docs |
| Agent Framework | LangGraph create_react_agent | ReAct loop with proper state management |
| Reasoning LLM | gpt-4o-mini (OpenAI) | Fast, cheap: reasoning and tool decisions |
| Report Writer | claude-sonnet-4-6 (Anthropic) | High prose quality for stakeholder-facing reports |
| Vector Store | Pinecone (media-inclusivity-index) | 19 vectors, 1536 dimensions, similarity threshold 0.70 |
| Embeddings | text-embedding-ada-002 (OpenAI) | Converts queries to vectors for RAG retrieval |
| Code Quality | Ruff linter (ruff.toml) | PEP8, unused imports, import ordering |

### 4. FastAPI: The Agent's Front Door

FastAPI is a Python web framework that turns Python functions into HTTP endpoints. It was chosen over the brief's suggested Execute Command node approach for three reasons:

- No cold start:  FastAPI stays running between requests. Execute Command re-starts Python each time.
- Input validation: Pydantic models reject malformed input before the agent even starts.
- Auto-documentation: FastAPI generates a full Swagger UI at localhost:8000/docs.




*FastAPI Swagger UI at localhost:8000/docs: auto-generated from Pydantic models, shows all endpoints and schemas*

### Pydantic Models

Pydantic enforces exact input/output shapes. If N8N sends malformed data, FastAPI rejects it immediately with a clear error — the agent never even starts.

```
class AnalyseRequest(BaseModel):
    company: str          # required — the outlet name
    focus: str = "all"    # optional — defaults to all communities

class ResearchResponse(BaseModel):
    status: str           # "success" or "error"
    overall_score: float  # 0.0 – 10.0
    harm_flags: list      # ⚠️ lines from report
    community_scores: dict  # {Women: 5.0, LGBTQ+: 3.0, ...}
```

### Score Parsing Bug Fix

The _parse_report_metrics() function extracts structured data from the Markdown report for Notion. A regex bug caused Notion to always show score 0.

- Problem: The regex was not matching bold Markdown like **Overall Average** | **3.25/10** — the ** markers were missing from the pattern.
- Fix: Updated regex to handle optional ** wrappers. Added fallback: if Overall Average row is not found, compute average from the four community scores.

*→ Lesson: Regex against LLM-generated text is fragile. Always add a fallback computation.*

### 5. ngrok: Bridging Cloud N8N to Local FastAPI

The original project plan assumed N8N Desktop was installed on the same Mac as FastAPI. In that setup, localhost:8000 is reachable directly. In the actual setup, N8N runs in the cloud: a cloud server cannot reach localhost on your laptop.

| Setup | Can N8N reach FastAPI? | ngrok needed? |
|---|---|---|
| N8N Desktop (same Mac) | Yes — both on same machine | No |
| N8N Cloud (app.n8n.cloud) | No — different machines | Yes |

ngrok runs on your Mac and creates a secure HTTPS tunnel. Any request to the ngrok URL is forwarded to local port 8000. N8N calls the ngrok URL: it never needs to know about localhost.

> *→ Important: the ngrok URL changes on every restart (free plan). After every ngrok restart, paste the new URL into the N8N HTTP Request node.*

> *→ Lesson: Always check whether components are cloud or local before building the integration.*

### 6. RAG System: Four Evaluation Angles

Without RAG the agent can describe what it found: e.g. 'NYT mentioned women 4 times today'; but cannot judge whether that is good, bad, or average. With RAG: '14% female bylines vs industry average of 37% (Nieman Lab 2023) —> a 23 percentage point gap.' That is a finding with a citation.

*Pinecone media-inclusivity-index — live, 19 vectors, 1536 dimensions, AWS us-east-1, On-demand capacity*

#### RAG Quality Test Results — All 4 Angles Pass (threshold: 0.70)

| Angle | Best Score | Threshold | Status |
|---|---|---|---|
| Bylines and Story Selection | 0.8981 | 0.70 | ✓ PASS |
| Portrayal Within Content | 0.8920 | 0.70 | ✓ PASS |
| Sourcing Diversity | 0.9024 | 0.70 | ✓ PASS |
| Language and Framing | 0.8487 | 0.70 | ✓ PASS |

### 7. The Four MCP Tools

MCP (Model Context Protocol) is the standardized pattern for giving an AI agent access to external data. Each tool is a Python function decorated with @tool from LangChain. The agent reads the docstring to decide when to call each tool: the docstring IS the instruction.

#### Tool 1: analyse_rss_feed: Primary Source

The only tool that reads what the outlet really publishes. Analyses the last 30 articles from each outlet's RSS feed.

- Guardian routing: detects 'guardian' → calls Guardian Open Platform API instead of RSS. Returns full article body text + bylines.
- NPR multi-feed: single /1001 feed gave 10 articles with no disability/LGBTQ+ coverage. Fixed by adding _fetch_npr_combined() that fetches 5 feeds (News + Politics + Business + Race + Health), deduplicates by URL, returns 30 unique articles with 27 named bylines.

- Thin article fallback: if RSS summary is under 150 chars, _fetch_article_text() fetches the article URL directly with a 5s timeout.

  *→ Why this matters: primary source evidence (what they publish) is more defensible than secondary reporting (what others say about them).*

### Tool 2: search_newsapi: External Perspective

Searches hundreds of news sources for recent articles about the outlet's diversity record (last 30 days, top 5). Captures criticism, awards, controversies.

### Tool 3: search_wikipedia: Background Context

Retrieves company overview, ownership, editorial history, stated commitments. Free; no API key required.

### Tool 4: rag_query_tool: Benchmark Retrieval

Wraps rag_pipeline.rag_query() as a @tool. Agent calls this 4+ times per run, once per angle. Converts query to a 1536-dimension vector, searches Pinecone, returns top 3 chunks above the 0.70 threshold.

*→ At 4 tools, token usage per run is ~2,850 tokens (2% of the 128k window). More text-heavy tools compound faster and risks context rot.*

## Known Limitations

The following are documented as designed constraints, not failures.

They reflect deliberate choices about scope, cost, and what counts as credible evidence.

| Limitation | Why It Exists | Honest Framing |
|---|---|---|
| RSS provides headlines and short summaries only (non-Guardian outlets) | RSS is a distribution format: outlets publish truncated content by design | Surface signal only. The agent qualifies RSS-based claims and uses them primarily for byline counting and language flags, not full content analysis |
| NYT article fetch returns paywall/login pages | NYT requires a paid API for full article access. URL fetching returns the login HTML, not article text | NYT still contributes named bylines and Wikipedia background. The paywall is the real ceiling: not a tool failure. This is documented in every NYT report |
| RAG knowledge base covers 18 documents | Built from publicly available journalism research and standards documents | 19 vectors is a working starting point. Scores should be read as directional, not definitive, until the knowledge base is expanded. |
| Full article text requires paid APIs, except Guardian | NewsAPI, NYT, and others charge for full-text access. The Guardian free key is an exception | Guardian benefits most from its full-text API. For other outlets, the agent relies on RSS summaries and secondary coverage. Every report source list makes this explicit |

*→ The through-line in all tool decisions: make the evidence as strong as possible without adding tools or complexity, and document limitations honestly.*

## 8. ReAct Pattern and LangGraph

### The ReAct Bug and the Fix

The original research prompt listed STEP 1 / STEP 2 / STEP 3 / STEP 4 with exact tool calls. The LLM read this as a shopping list and fired all tools simultaneously in one batch: Total messages: 11 | Tool calls: 1.

- Fix: Rewrote _build_research_prompt() to say "Call tools ONE AT A TIME. After each tool result, stop and reason before deciding your next action."
- Added one explicit starting action: "BEGIN: Call analyse_rss_feed() now".
- Then gave adaptive examples: "if RSS shows no disability coverage → query RAG for disability standards specifically".

Result: Total messages: 20–30 | Tool calls: 6–10+. The agent now adapts its research based on what it finds — true ReAct behaviour.

> *→ Lesson: Prompt design is agent design. A to-do list format bypasses ReAct entirely. One starting action + adaptive instructions produces real reasoning loops.*

### Why create_react_agent Instead of a Custom 5-Node Graph

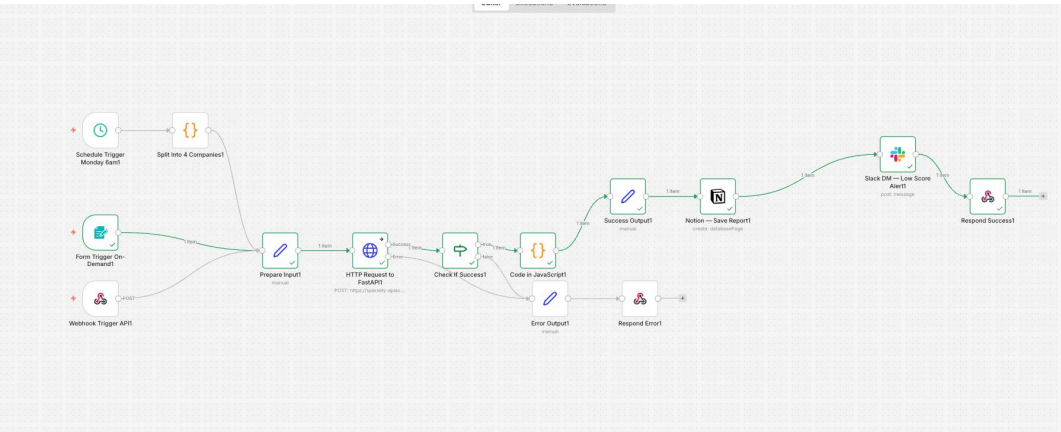
The rubric asks for a LangGraph workflow. LangGraph's prebuilt create_react_agent builds a full StateGraph internally: the requirement is fully satisfied. A fixed graph calls tools in hardcoded order regardless of what is found.

create_react_agent lets the agent decide: it reads what it has found and decides what to look up next.

### Why Two LLMs

| Nodes | Model | Why |
|---|---|---|
| PLAN / RESEARCH / ANALYSE / SYNTHESISE | gpt-4o-mini | Fast, cheap: good at structured reasoning and tool decisions |
| REPORT | claude-sonnet-4-6 | Report prose quality matters when stakeholders read it |

## 9. N8N Workflow — Entry, Exit, and Changes Made

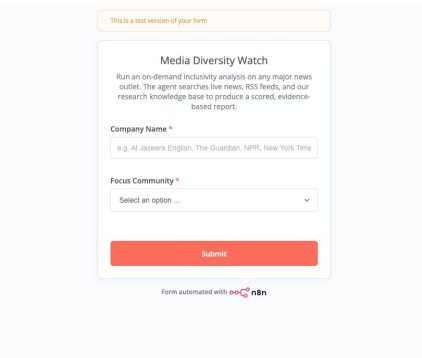



*N8N workflow canvas — Schedule Trigger, Form Trigger, Webhook, HTTP Request to FastAPI via ngrok, Notion, Slack DM*

### Four Triggers

| Trigger | When it fires | Use |
|---|---|---|
| Schedule Trigger | Monday 6am automated | Primary — weekly autonomous run |
| Form Trigger | Researcher submits web form | On-demand — type any company, get report |
| Webhook Trigger | HTTP POST from external code | API integration from scripts |
| Manual Trigger | Click 'Test' in N8N canvas | Demo and development testing |

### On-Demand Form




*Media Diversity Watch on-demand form — enter any outlet name and focus community*

### N8N Changes Made

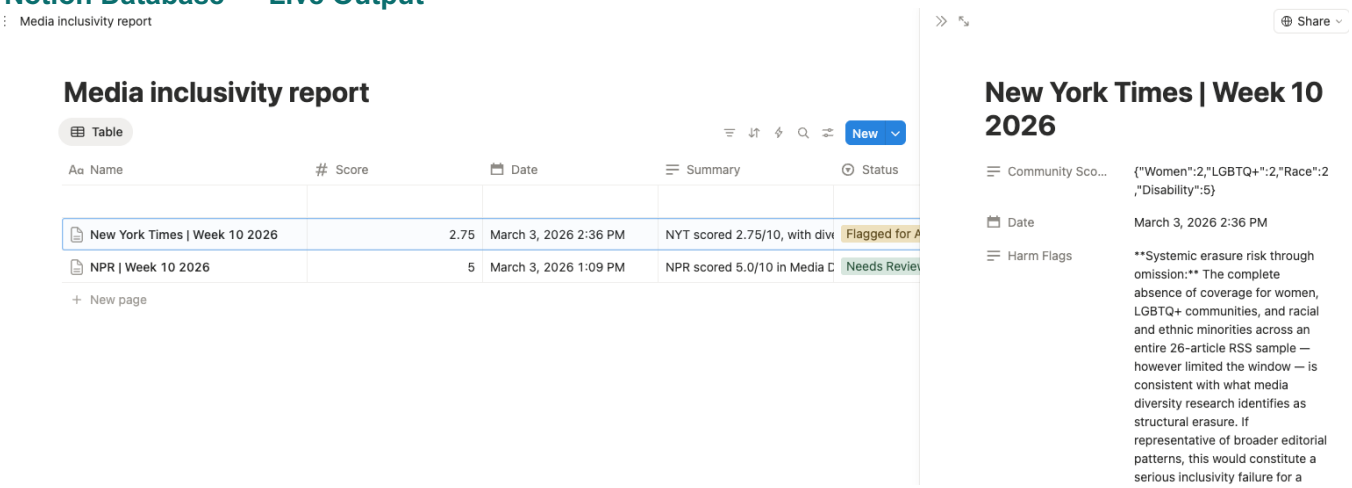
| Change | Why |
|---|---|
| Fixed overall_score in Success Output — was empty {} | Notion was receiving no score value |
| Fixed Slack field names $json.* → $(Success Output).first().json.* | After Notion node, $json = Notion API response, not our data |
| Added 2 extra Notion blocks (7→9) | Reports up to 15,303 chars were getting cut off at 13,300 |
| Removed Score >= 7? and Score >= 4? IF nodes — single Notion node | Simpler, less error-prone; dynamic Status field replaces 3 paths |
| Dynamic Status: score >= 7 ? Good Standing : >= 4 ? Needs Review : Flagged for Action | Single node replaces three conditional Notion nodes |
| Slack DM fires on every run (was: only score < 4) | Researcher wants notification every time |
| Fixed Split Into 4 Companies mode: runOnceForEachItem → runOnceForAllItems | Was causing 'json property isn't an object' error on schedule trigger |
| YYYY → yyyy in date formatting token | YYYY = ISO week-year (wrong). yyyy = calendar year (correct). N8N uses Luxon. |

## 10. Generated Report Structure and Live Outputs

Every report is generated autonomously by claude-sonnet-4-6. The 11-section structure ensures consistency across all outlets and all weeks.

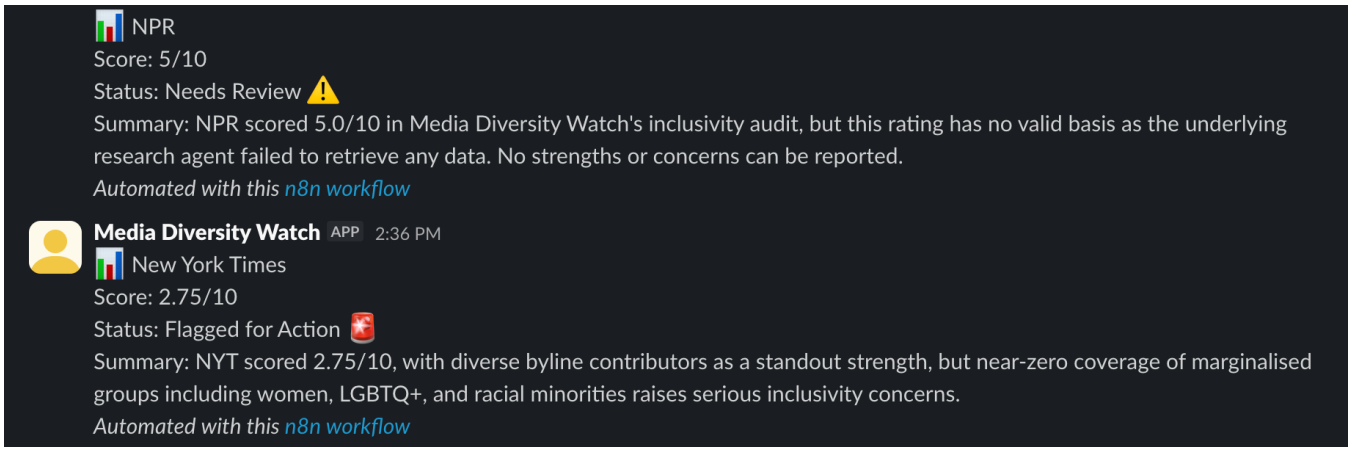
| Section | Title | Sources Used |
|---|---|---|
| 1 | Company Overview | Wikipedia — ownership, history, stated commitments |
| 2 | Angle 1: Bylines and Story Selection | RSS / Guardian API — > who is writing stories? |
| 3 | Angle 2: Portrayal Within Content | NewsAPI + RAG — > how are communities framed? |
| 4 | Angle 3: Sourcing Diversity | NewsAPI + RAG — > whose knowledge is authoritative? |
| 5 | Angle 4: Language and Framing | RSS + RAG — > inclusive vs harmful language audit |
| 6 | Community-by-Community Findings | All tools — > Women, LGBTQ+, Race, Disability separately |
| 7 | Key Strengths | Positive findings with exact source citations |
| 8 | Harm Flags | Non-inclusive language or framing with exact evidence |
| 9 | Inclusivity Scores | 1-10 per community with methodology explanation |
| 10 | Recommendations | Specific, actionable, evidence-based suggestions |
| 11 | Full Source List | Every claim sourced to the tool that retrieved it |

## Notion Database — Live Output




*Notion database showing The New York Times | Week 10 2026*

### Slack Alerts




*Slack DM showing NYT 2.75 /10 Flagged for Action and 5.0/10 Needs Review — automated via N8N*

### 11. Anti-Hallucination Architecture

This agent makes claims about real companies. A hallucinated score or invented statistic could damage a real organization's reputation. Four layers prevent this:

- Layer 1 — Grounded Prompts: every LLM call instructs the agent to only make claims supported by tool results. If no evidence exists, state "No evidence found."
- Layer 2 — Source Citations Required: every finding must cite its source (NewsAPI, Guardian API, Wikipedia, RSS, or Pinecone RAG). Unsourced claims are blocked.
- Layer 3 — Pydantic Schema Validation: every analysis call returns structured JSON with defined required fields. Forces specificity, prevents vague filler text.
- Layer 4 — RAG as Ground Truth: the Pinecone knowledge base provides verified inclusivity benchmarks. Agent compares findings against real research, not its own assumptions. Similarity threshold 0.70 means only strongly relevant benchmarks are used.

### 12. Challenges, Solutions and Reflections

| Challenge | What Happened | Solution | Reflection |
|---|---|---|---|
| ReAct wasn't actually iterating | STEP 1/2/3/4 prompt caused agent to fire all tools at once. 11 messages, 1 tool call cycle. | _build_research_prompt() rewritten — "ONE AT A TIME", 20–30 messages, 6–10+ calls. | Prompt design is agent design. A to-do list bypasses ReAct entirely. |
| All outlets scoring 5.0/10 | Thin, undifferentiated evidence meant every outlet got the same score. | Added data source constraints — RSS = surface signal only, RAG = benchmarks not proof. Agent must qualify claims by evidence strength. | Undifferentiated scores are worse than no scores — they erode trust in the system. |
| N8N score always showing 0 | Regex in _parse_report_metrics() didn't match **Overall Average** (bold Markdown). Notion received score: 0. | Regex updated for optional ** markers. Fallback added: compute average from community scores if Overall Average row not found. | Regex against LLM-generated text is fragile. The fallback approach is more robust for production. |

| Challenge | What Happened | Solution | Reflection |
|---|---|---|---|
| N8N localhost unreachable | Original plan assumed N8N Desktop. Actual setup uses N8N Cloud. Cloud can't reach localhost. | Added ngrok tunnel. n8n/README.md fully rewritten to document correct setup. | Always check whether components are cloud or local before building the integration. |
| NPR weak evidence | Single /1001 feed returned 10 articles with no disability or LGBTQ+ content. | _fetch_npr_combined(): 5 topic feeds (News, Politics, Business, Race, Health) → 30 unique articles, 27 named bylines. | Topic-specific feeds are significantly more valuable for an inclusivity audit than a single general feed. |
| Notion report truncated | Reports up to 15,303 chars exceeded 13,300 char limit (7 blocks × 2000 chars). | Added 2 extra blocks to Good Standing + Needs Review (7→9). 1 extra to Flagged for Action. | Always test with the longest expected output, not average output. |
| Date token YYYY vs yyyy | N8N title showed 'Week 10 YYYY' — YYYY is ISO week-year, not calendar year. | Changed 'YYYY' → 'yyyy'. N8N uses Luxon where yyyy = 4-digit calendar year. | Luxon's tokens are case-sensitive and differ from JavaScript Date conventions. |
| Scores were uniformly 1–2/10 across all outlets | The scoring prompt had no distinction between "active harm found" and "no data available." RSS feeds return only 105–130 character teasers — when the agent found no community keywords, it scored 1–2 by default, the same band as confirmed harmful content. | Updated the scoring guide in report_generator.py to reserve 1–2 for actively found harm only. Added a hard rule: *absence of evidence is not evidence of harm.* Thin RSS data now scores 4–5 with a data limitation note, while The Guardian (full article API) retains higher scores — making the gap between outlets meaningful. | Evaluation prompts need explicit rules for incomplete data, not just rules for good and bad outcomes. Every ambiguity in a scoring rubric gets amplified — the model always picks the most conservative interpretation. When data is thin, you need to tell it what to do, or it will penalise the outlet for what it couldn't see. |

### 13. Quick Reference

| Number | Meaning |
|---|---|
| 4 | Outlets: Al Jazeera English, The Guardian, NPR, New York Times |
| 4 | MCP tools: RSS/Guardian API, NewsAPI, Wikipedia, Pinecone RAG |
| 4 | Evaluation angles: Bylines, Portrayal, Sourcing, Language |
| 4 | Communities: Women, LGBTQ+, Race, Disability |
| 5 | LangGraph nodes: PLAN, RESEARCH, ANALYSE, SYNTHESISE, REPORT |
| 18 | Documents in RAG knowledge base |

| Number | Meaning |
|---|---|
| 19 | Vectors in media-inclusivity-index |
| 0.70 | Minimum RAG similarity score for evidence use |
| 0.87+ | Average RAG quality score across all 4 angles |
| ~2,850 | Token usage per agent run (2% of 128k window) |
| 11 | Report sections per outlet |
| 9 | Notion content blocks per report (supports ~18k chars) |
| 8000 | FastAPI port |
| 8001 | Mock server port (changed to avoid conflict) |
| 300s | N8N HTTP request timeout |

### Start Commands

```
python run.py          # Terminal 1 — start FastAPI on port 8000
ngrok http 8000        # Terminal 2 — create tunnel, copy URL to N8N
localhost:8000/docs    # FastAPI Swagger UI (when server is running)
ruff check src/ --fix  # Run before every commit
```